

Widget Master

Plan de desarrollo completo

App web de widgets de productividad

Junio 2026

Stack Tecnológico

Frontend	React 18 + Vite + TypeScript + Tailwind CSS
Backend	NestJS + TypeScript
ORM	Prisma
Base de datos	PostgreSQL
Monorepo	pnpm workspaces + Turborepo
Drag & Drop	@dnd-kit (freeform)
Autenticación	JWT (Fase 3)

Fase 1: Scaffolding

Objetivo: Inicializar el monorepo, configurar el backend con NestJS y Prisma, el frontend con Vite y React, y el paquete compartido de tipos.

Estructura de carpetas

```
widget-master/  
% % % apps/  
    % % % frontend/      # React + Vite + Tailwind  
    % % % backend/       # NestJS + Prisma  
% % % packages/  
    % % % shared/        # Tipos y DTOs compartidos  
% % % package.json       # pnpm workspaces  
% % % turbo.json         # Turborepo config  
% % % .prettierrc  
% % % .eslintrc.js
```

Tareas

- Inicializar pnpm workspaces con filtros apps/* y packages/*
- Configurar Turborepo con pipelines build, dev, lint
- Crear apps/backend con NestJS CLI, TypeScript, Prisma
- Crear apps/frontend con Vite + React + Tailwind CSS
- Crear packages/shared con tipos base (User, Widget, etc.)
- Configurar Prisma schema con modelos User, Widget, TodoList, TodoItem
- Ejecutar migración inicial de Prisma
- Verificar que los scripts dev funcionan en paralelo con turbo

Esquema de base de datos (Prisma)

```
model User {  
  id      String  @id @default(uuid())  
  email   String  @unique  
  password String  
  widgets Widget[]  
  lists   TodoList[]  
}  
  
model Widget {  
  id      String  @id @default(uuid())  
  userId  String  
  user    User    @relation(fields: [userId], references: [id])  
  type    String  
  config  Json  
  position Json  
  zIndex  Int      @default(0)  
}  
  
model TodoList {  
  id      String  @id @default(uuid())  
  userId  String  
  user    User    @relation(fields: [userId], references: [id])  
  title   String  
  items   TodoItem[]  
}  
  
model TodoItem {  
  id      String  @id @default(uuid())  
  listId  String  
  list    TodoList @relation(fields: [listId], references: [id])  
  text    String  
  completed Boolean @default(false)  
  order   Int  
}
```

Fase 2: Sistema de Widgets (Core)

Objetivo: Implementar el corazón de la aplicación: CRUD de widgets en backend y frontend con grid libre (freeform drag & drop).

Backend

- Módulo NestJS Widgets (Controller, Service, Module)
- DTOs de creación y actualización de widgets
- CRUD completo: GET /widgets, POST /widgets, PATCH /widgets/:id, DELETE /widgets/:id
- Validación con class-validator / pipes de NestJS

Frontend

- WidgetGrid: componente principal que renderiza todos los widgets
- @dnd-kit con DndContext, useDraggable y useDroppable (freeform)
- WidgetFrame: wrapper de cada widget con drag handle, botón de eliminar y resize
- Widget Registry: mapa de widgetType -> componente React
- Carga dinámica de widgets desde el registro
- Persistencia de posición: onDragEnd -> PATCH /widgets/:id

API endpoints

```
GET    /api/v1/widgets
POST   /api/v1/widgets
PATCH /api/v1/widgets/:id
DELETE /api/v1/widgets/:id
```

Fase 3: Autenticación

Objetivo: Agregar registro e inicio de sesión con JWT para que cada usuario tenga sus propios widgets.

Backend

- Módulo Auth NestJS con estrategia JWT
- Register: POST /auth/register (email + password hasheado con bcrypt)
- Login: POST /auth/login (devuelve JWT token)
- Guard de NestJS para proteger rutas
- Reemplazar userId hardcodeado por userId del token

Frontend

- AuthContext con React Context API
- Página de Login y Register
- Almacenar token en localStorage
- Interceptor de Axios para añadir Bearer token
- Proteger rutas con componente PrivateRoute
- Mostrar usuario logueado en la interfaz

Fase 4: Widgets Específicos

Objetivo: Implementar los widgets funcionales uno por uno.

4.1 Todo List

- CRUD de listas y items
- Checkbox para completar tareas
- Reordenar items con drag
- Autosave al editar texto

4.2 Pomodoro Timer

- Timer configurable (25min trabajo, 5min descanso)
- Estados: idle, running, paused, break
- Notificación al cambiar de estado
- Persistencia del estado actual

4.3 Notas

- Textarea con autosave (debounce)
- Soporte para markdown básico (opcional)
- Fecha de última modificación

4.4 Calculadora

- Operaciones básicas: +, -, *, /
- Historial de operaciones
- Persistencia del último resultado

Widget Registry

```
const widgetRegistry = {
  "todo-list": { component: TodoListWidget, icon: CheckSquare,
    label: "Lista de tareas", defaultConfig: {} },
  "pomodoro": { component: PomodoroWidget, icon: Clock,
    label: "Pomodoro", defaultConfig: { work: 25, rest: 5 } },
  "notes": { component: NotesWidget, icon: FileText,
    label: "Notas", defaultConfig: { content: "" } },
  "calculator": { component: CalculatorWidget, icon: Calculator,
    label: "Calculadora", defaultConfig: {} },
};
```

Fase 5: Polish y Publicación

Objetivo: Pulir la aplicación y dejarla lista para uso real.

Diseño responsive

- Adaptar WidgetGrid a pantallas pequeñas
- Widgets en columna única en mobile
- Sidebar/panel para añadir widgets

Tema claro/oscuro

- Contexto de tema en React
- Persistencia en localStorage
- CSS variables para colores
- Botón de cambio en la interfaz

PWA

- Manifest.json con iconos
- Service worker para caché de assets
- Instalable en escritorio y móvil
- Funcionamiento offline básico

Despliegue

- Frontend: Vercel o Netlify
- Backend: Railway, Render o Fly.io
- Base de datos: Supabase PostgreSQL o Neon
- Variables de entorno para producción
- CI/CD básico con GitHub Actions

¡Manos a la obra!

Widget Master — App web de widgets de productividad

Creado con OpenCode • Junio 2026